Key Design Decisions

A. How items advertise their available actions to the player

Design 1: Hardcode instanceof checks in ContractedWorker.playTurn() to detect each item type in the inventory and manually build the action list (the original approach).

Pros	Cons
Simple and straightforward to implement, all action logic is in one place.	Violates the Open-Closed Principle; every new item requires modifying ContractedWorker.
Easy to understand the flow of action generation by reading a single method.	Creates tight coupling between ContractedWorker and every concrete item class (Flask, AccessCard, etc.), reducing maintainability.
	Duplicates functionality already provided by the engine's World.prepareAllowableActions(), which calls Item.allowableActions() automatically each turn.

Design 2: Each Item subclass overrides allowableActions(Actor, GameMap) to return its own actions. The engine's World.prepareAllowableActions() automatically collects these and passes them to playTurn().

Pros	Cons
Follows the Tell, Don't Ask principle; each item is responsible for its own behaviour rather than being interrogated by the actor.	Action logic is distributed across multiple classes, which may be harder to trace for someone unfamiliar with the engine's game loop.
New items can be added without modifying ContractedWorker at all, satisfying the Open-Closed Principle.	Items now have a dependency on their corresponding Action classes.
Leverages the engine's existing infrastructure, avoiding code duplication (DRY).	

B. How the door unlocking mechanism works

Design 1: Use instanceof AccessCard checks in ContractedWorker and compare the ground's display character (== '=') to find doors (the original approach).

Pros	Cons
Quick to implement with minimal classes involved.	Checking display characters is fragile; if the character changes, the logic breaks silently.
	ContractedWorker becomes tightly coupled to both AccessCard and Door concrete classes.
	Adding a second item that can open doors (e.g., a Master Key) would require modifying ContractedWorker again, violating the Open-Closed Principle.

Design 2: Use the engine's ability system. AccessCard enables Ability.CAN_UNLOCK_DOORS on itself. Door overrides Ground.allowableActions() and checks actor.hasAbility(Ability.CAN_UNLOCK_DOORS) to offer an UnlockDoorAction targeting that specific door.

Pros	Cons
Door and AccessCard are decoupled from each other; both depend on the abstract Ability enum instead (Dependency Inversion Principle).	Requires creating an additional enum class (Ability), adding a small amount of structural overhead.
Any future item that enables CAN_UNLOCK_DOORS will work with all doors without modification (Open-Closed Principle).	
Each UnlockDoorAction targets a specific Door instance, avoiding the original bug where the action would unlock an arbitrary adjacent door.	

C. How the FirstAidKit cooldown timer works

Design 1: Track cooldown using a counter that is decremented every turn by the game loop or by ContractedWorker.playTurn(), regardless of whether the kit is being carried or on the ground.

Pros	Cons
Simple to implement; a single counter decremented each turn.	Does not satisfy the requirement that the timer only progresses while carried. Would need additional checks to determine if the item is in an inventory or on the ground.
	Requires ContractedWorker to know about FirstAidKit internals, increasing coupling.

Design 2: Override Item.tick(Location, Actor) (called only while carried) to decrement the cooldown. Do not override Item.tick(Location) (called while on the ground), so the timer naturally freezes when the kit is dropped.

Pros	Cons
Directly satisfies the specification that the cooldown only progresses while carried, using the engine's built-in lifecycle distinction between the two tick() overloads.	Relies on understanding the engine's tick mechanism — the distinction between tick(Location, Actor) and tick(Location) is not immediately obvious.
No external class needs to manage the timer; FirstAidKit is fully self-contained (Single Responsibility Principle).	
FirstAidKit.allowableActions() only offers UseFirstAidKitAction when isReady() returns true, keeping cooldown logic encapsulated.	

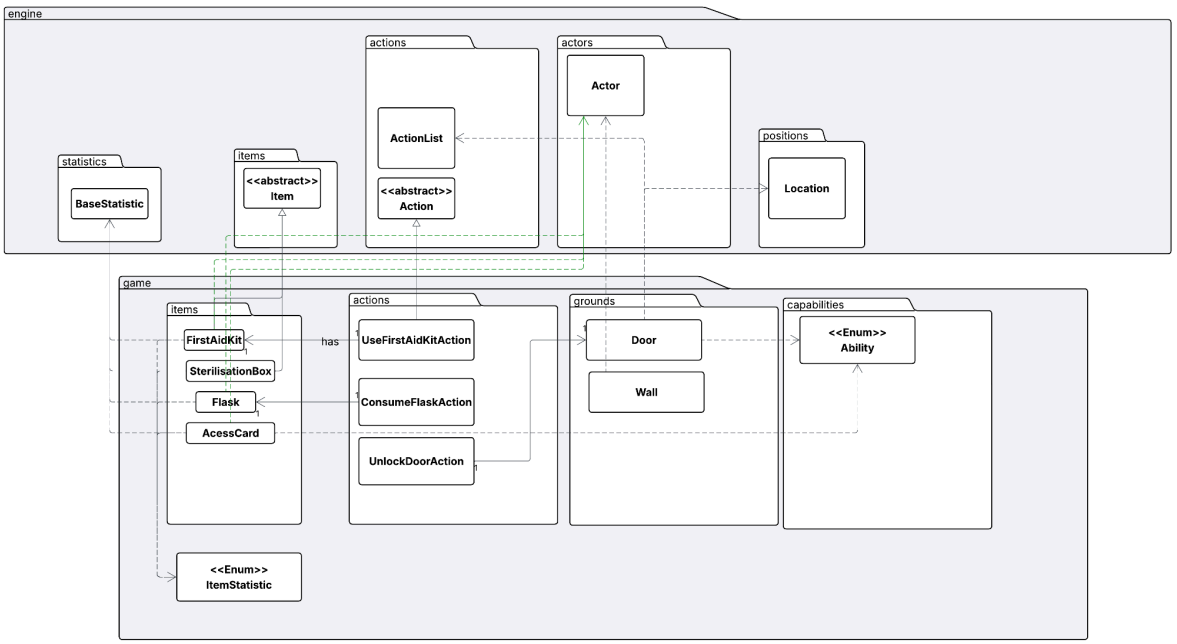
D. How worker inventories are managed

Design 1: Create a single BasicInventory instance and share it across all workers (the original approach).

Pros	Cons
Fewer objects created; marginally simpler initialisation code.	Critically incorrect: all workers see the same items, so picking up an item as one worker makes it appear in every worker's inventory.
	BasicInventory has no weight limit, so the 50-unit constraint from the specification is not enforced.

Design 2: Each worker receives their own WeightLimitedInventory(50) instance, created via a createWorker() factory method in EclipseNebula.

Pros	Cons
Correctly isolates each worker's inventory, matching the specification for individual 50-unit weight limits.	Slightly more memory usage with five separate inventory objects.
The factory method createWorker() centralises worker setup (name, inventory, flask), avoiding repeated code (DRY).	
WeightLimitedInventory checks ItemStatistics.WEIGHT on every add/remove, enforcing the weight constraint through the engine's statistics system rather than ad-hoc fields.	



Summary

The final design of requirement 1 utilises Design 2 of parts A, B, C, and D stated above to implement the contracted workers, their inventories, the item action system, and the door-unlocking mechanism.

All new item classes (AccessCard, Flask, FirstAidKit, SterilisationBox) extend the engine's abstract Item class, inheriting common attributes such as name, display character, and the statistics/ability framework. This avoids repeating shared functionality across items (DRY). Each item registers its weight via addNewStatistic(ItemStatistics.WEIGHT, ...), reusing the engine's extensible statistics system rather than introducing ad-hoc weight fields.

Items are responsible for advertising their own actions by overriding allowableActions(Actor, GameMap). Flask returns a ConsumeFlaskAction when uses remain; FirstAidKit returns a UseFirstAidKitAction when off cooldown. This delegates action generation to the items themselves rather than centralising it in ContractedWorker, meaning ContractedWorker.playTurn() contains no item-specific logic. Each class has a single, clear responsibility (Single Responsibility Principle), and new items can be introduced without modifying existing classes (Open-Closed Principle).

The door-unlocking mechanism uses the engine's ability system as an abstraction layer. AccessCard enables Ability.CAN_UNLOCK_DOORS on itself, and Door checks for this ability when deciding whether to offer an UnlockDoorAction. Neither class depends on the other directly; both depend on the Ability enum (Dependency Inversion Principle). The UnlockDoorAction receives a reference to the specific Door and its Location, ensuring deterministic behaviour when multiple doors are adjacent.

FirstAidKit's cooldown leverages the engine's two distinct tick() overloads: tick(Location, Actor) for carried items and tick(Location) for grounded items. By only overriding the carried variant, the cooldown timer naturally freezes when the kit is on the floor, satisfying the specification without any external timer management.

Each worker is constructed with an independent WeightLimitedInventory(50) via the createWorker() factory method, ensuring correct isolation and consistent setup across all five workers.